Project Proposal: Unified Detection of Tenant-Isolation Violations and QoS Fairness Breaches in Multi-Tenant Cloud Platforms

1. Base Paper

Title: Dynamic Application Security Testing for Kubernetes Deployment: An Experience Report from Industry

Authors: Shazibul Islam Shamim, Hanyang Hu, Akond Rahman

Venue: 33rd ACM International Conference on the Foundations of Software Engineering (FSE '25), Trondheim, Norway, June 2025

DOI / Link: https://doi.org/10.1145/3696630.3728573

PDF: https://akondrahman.github.io/files/papers/fse25.pdf

---

2. What the Paper Does

The paper is an industry experience report examining the use of Dynamic Application Security Testing (DAST) to detect security-relevant configuration violations in Kubernetes deployments. Working with an industry partner ("Company-Z"), the authors evaluate three DAST tools against a live Kubernetes environment and find thousands of CIS-recommended configuration violations. The authors recommend that practitioners adopt DAST tooling as a complement to static analysis, arguing that static checks alone cannot catch violations that only manifest once a cluster is running under real workloads.

The paper's contribution is empirical and advisory: it documents observations from a real-world deployment and derives a practitioner-facing recommendation. It treats Kubernetes as a generic deployment substrate rather than engaging with the defining property of **cloud computing that Kubernetes is usually used to deliver: multi-tenancy and shared resource pooling.**

---

3. Limitations of the Base Paper

1. **No concrete implemented artifact.** The paper documents findings and recommendations but does not design, build, or release a detection tool.
2. **Not framed around cloud computing's core value proposition.** Multi-tenancy and resource pooling are foundational, defining characteristics of cloud computing (this is how a cloud provider serves many customers efficiently from shared infrastructure). The paper does not engage with this at all — it treats "configuration violation" as a generic concern, not one tied to the thing that makes cloud computing economically and architecturally distinct: **many tenants safely and fairly sharing one pool of resources.**
3. **Single-dimension notion of "violation."** The paper only considers security misconfiguration. It does not consider that in a multi-tenant cloud setting, a shared

platform can fail its tenants in two related but distinct ways: (a) **isolation failures** — one tenant improperly *accessing* another tenant's resources (a security problem), and (b) **fairness failures** — one tenant improperly *starving* another tenant of shared resources, degrading their QoS/SLA (a resource-management problem). Both stem from the same root cause — imperfect enforcement of tenant boundaries — but the paper, and most follow-on work, treats only the security half.

4. **No quantitative evaluation.** The paper reports no precision, recall, false-positive rate, or detection latency, and does not compare dynamic detection against a static baseline in a controlled way.
5. **Single signal source.** Dynamic detection approaches in this space typically rely on control-plane signals (API/audit activity) alone, missing violations that occur purely at the data/resource plane.

---

4. Limitations We Are Addressing (Our Novelty)

We address Limitations 1, 2, 3, and 4 directly, with Limitation 2 and 3 forming the heart of our cloud-computing framing:

- **Against Limitation 1 (no implemented artifact):** We design, implement, and demonstrate a working monitoring system rather than only recommending the general approach.
- **Against Limitation 2 (not framed around cloud's core value proposition):** We put **multi-tenancy and resource pooling** — the defining cloud computing concept — at the center of the project. Our system is explicitly built around the question a cloud provider actually has to answer: *"Are my tenants safely and fairly sharing this pool of infrastructure?"* rather than the generic question "is this cluster misconfigured?"
- **Against Limitation 3 (security-only notion of violation):** This is our central novelty. We treat **tenant isolation** (security: did Tenant A improperly access Tenant B's resources?) and **tenant fairness/QoS** (resource management: did Tenant A's workload degrade Tenant B's promised performance — the classic cloud computing "noisy neighbor" problem?) as two faces of the same underlying monitoring problem, and build a **single unified detection pipeline** that reports both. This directly engages core cloud computing concepts — elasticity, resource pooling, QoS/SLA guarantees — rather than treating the cloud platform as an incidental deployment target for a security tool.
- **Against Limitation 4 (no quantitative evaluation):** We evaluate against a structured library of injected isolation-violation *and* fairness-violation scenarios, reporting precision, recall, false-positive rate, and detection latency for each category, compared against a static-only baseline.
- **Against Limitation 5 (single signal source):** We fuse **control-plane** signals (Kubernetes audit logs — who asked to access what) with **data-plane/resource-plane** signals (eBPF-based network flow and resource-consumption telemetry — what actually happened on the wire and on the node), so that both access violations (visible

mainly in traffic) and fairness violations (visible mainly in resource consumption) are caught by the same pipeline.

Novelty statement: The base paper argues, generically, that dynamic detection should complement static configuration review in Kubernetes. We reframe this recommendation around cloud computing's defining property — multi-tenant resource pooling — and extend it to cover both halves of what "safe multi-tenancy" actually requires: tenants not accessing each other's resources (isolation) and tenants not starving each other's resources (fairness/QoS). We back this unified view with a quantitative evaluation the base paper does not provide.

---

5. Proposed Approach

5.1 System Overview

We will build a small multi-tenant SaaS application, containerized with Docker and deployed on Kubernetes, with tenants isolated in separate namespaces via RBAC, NetworkPolicy, and resource quotas (CPU/memory limits per tenant). On top of this, we build a **unified tenant-health monitoring layer** with two detection tracks sharing one telemetry pipeline:

1. **Static baseline:** A one-time compliance check using kube-bench against CIS Kubernetes benchmarks, representing the "static-only" approach — for both security misconfiguration and basic resource-quota misconfiguration.
2. **Control-plane monitor:** Consumes Kubernetes audit logs in near real time, flagging any principal from one tenant's namespace accessing a resource scoped to a different tenant.
3. **Data/resource-plane monitor:** Uses eBPF (e.g., via Cilium/Hubble) to capture two things from the same low-level vantage point:
    - **Network flows** crossing tenant namespace boundaries (isolation track).
    - **Resource consumption patterns** (CPU throttling, memory pressure, node-level contention) attributable to a specific tenant's workload spilling into another tenant's allotted share (fairness/QoS track).
4. **Fusion and classification layer:** Correlates control-plane and data/resource-plane events by namespace, principal, and timestamp, and classifies each detected anomaly as an **isolation violation** (security), a **fairness violation** (QoS/noisy-neighbor), or both.
5. **CI/CD security gate:** A pipeline stage scanning container images for known vulnerabilities before deployment.

5.2 Evaluation Methodology

We construct two parallel scenario libraries, injected into the test cluster in a controlled, repeatable manner:

- **Isolation scenarios:** network policy misconfiguration, RBAC over-permissioning, credential misuse, privilege escalation, data-plane-only boundary crossings.

- **Fairness/QoS scenarios:** a tenant workload deliberately over-consuming shared CPU/memory/I/O (e.g., a runaway batch job or resource-quota bypass), causing measurable latency or throughput degradation for a co-located tenant.

For each scenario, we record whether the static baseline, the control-plane monitor, and the full unified monitor detect the issue, and how quickly. For fairness scenarios specifically, we additionally measure the **QoS impact avoided** — e.g., latency/throughput degradation for the victim tenant with vs. without timely detection and (optionally) automated remediation such as re-applying resource quotas.

Aggregate precision, recall, false-positive rate, and detection latency are computed separately for the isolation track and the fairness track, and jointly for the unified pipeline, to show that a single monitoring investment covers both cloud computing failure modes rather than requiring two separate systems.

---

6. Feasibility

The proposed system relies entirely on established, freely available open-source tooling, requiring no paid cloud infrastructure:

Component	Tooling
Containerization	Docker
Orchestration	Kubernetes (local cluster via kind or Minikube)
Image vulnerability scanning	Trivy
Static configuration baseline	kube-bench
Control-plane monitoring	Kubernetes audit logging + custom detection logic
Data/resource-plane monitoring	eBPF via Cilium/Hubble (network flows + resource metrics)
Resource quota enforcement	Kubernetes ResourceQuotas / LimitRanges
Load generation (for fairness scenarios)	k6 or a simple custom load generator
CI/CD	GitHub Actions

Cilium/Hubble provides ready-made eBPF-based network *and* resource observability, meaning the same data collection layer can feed both detection tracks without building two separate telemetry systems — this is what keeps the unified design feasible within a short project timeline. Each component (static baseline, control-plane monitor, data-plane monitor, fusion layer, and the two scenario libraries) can be developed and tested incrementally and independently, so the evaluation can proceed even if any single piece

needs extra iteration. The full system is achievable within a short academic project timeline using a small team, with no dependency on paid or restricted resources.

---

7. Related Work Note

Existing eBPF-based Kubernetes security tools (e.g., Falco, Cilium/Tetragon) provide general-purpose runtime anomaly detection, and separate lines of work address cloud resource fairness and noisy-neighbor mitigation. Recent 2025 work has surveyed eBPF-based Kubernetes security tooling and explored federated eBPF-based anomaly detection, but these efforts treat security and resource-fairness monitoring as separate concerns with separate pipelines. No identified IEEE/ACM publication unifies **tenant-isolation security detection** and **tenant fairness/QoS detection** under a single fused control-plane/data-plane pipeline, evaluated quantitatively across both failure modes. This is the gap our project fills, and it is what puts cloud computing's defining concept — safe, fair multi-tenant resource pooling — at the center of the contribution rather than at its periphery.